SQLaw: Detecting Bugs in GPU Database Management Systems via Rule-Based Differential Execution

Jiaxin Hu and Rongxin Wu

***Abstract*—Database Management Systems (DBMSs) are essential for managing structured data. To meet the increasing performance requirements for complex, large-scale data management and analysis, GPU DBMSs have been introduced to enhance processing and query execution speeds. Despite the growing interest in GPU DBMSs and the inherent presence of bugs, there has been no systematic effort, to our knowledge, to detect bugs in GPU DBMSs. To this end, we design SQLaw, an innovative and comprehensive framework that combines offline rule learning with an online interpreter incorporating mutation for efficient and general GPU-related bug detection. The offline rule learning component automatically extracts differential execution rules, which are used to guide the synthesis of configuration and query statements for testing. The online interpreter with mutation ensures the generalization of these statements. We evaluated SQLaw on three major GPU DBMSs. Our extensive evaluations demonstrate that SQLaw outperforms current state-of-the-art approaches by up to 2.22× in the number of bugs detected within 24 hours. Additionally, SQLaw detected 51 previously unknown GPU-related bugs, of which 37 have been confirmed or fixed by developers.**

***Index Terms*—GPU DBMSs, bug detection, differential execution rule.**

I. Introduction

DATABASE management systems (DBMSs) are fundamental components in numerous enterprise software. GPU DBMSs, as an essential branch of DBMSs, are playing an increasingly significant role with the advent of the big data era [1]. With the powerful parallel computing and high-bandwidth memory access capabilities of GPUs [2], [3], [4], [5], GPU DBMSs can accelerate data processing, query execution, and more, to meet surging performance demands for increasingly complex and large-scale data management and analysis. Meanwhile, GPU DBMSs are widely applied in industries such as communications, finance, transportation, and manufacturing, and others [6], [7], [8], [9]. However, due to the complexity of GPU features, implementing GPU-related applications correctly is non-trivial, and thus critical bugs are inevitable [10], [11], [12]. Unfortunately, to the best of our knowledge, there has yet to be a systematic study to detect bugs in GPU DBMSs.

Essentially, detecting GPU-related bugs in GPU DBMSs entails tackling two fundamental problems. The first problem is to come up with an effective test oracle that can verify whether a GPU DBMS behaves correctly for a given input. Inspired by the widely adopted differential testing approach for bug detection in non-GPU DBMSs [13], [14], [15], [16] and GPU-related software [17], [18], we borrow its core concept of comparing the results of identical test cases executed on different software systems to establish a test oracle [19]. However, directly applying the aforementioned differential testing approaches may not be applicable in detecting GPU-related bugs in DBMSs. First, the existing approaches for non-GPU DBMSs are limited to detecting bugs in non-GPU-specific features [13], [14], [15], [16]. For example, *Grand* [13] leverages multiple Gremlin-based graph databases to verify the return results of same queries, thus inherently limiting its applicability to other database paradigms. *DT2* [15] limits its effectiveness for identifying transaction-level bugs in databases. Similarly, *THANOS* [14] maintains its detection efficacy only when applied to databases incorporating multiple storage engines. Second, some studies on testing GPU-related software (e.g., deep learning frameworks) crosscheck the execution results under different counterpart framework APIs [18] to find bugs. However, adapting such an approach to GPU DBMSs faces significant limitations due to substantial dialectal variations among GPU DBMS implementations, including divergent grammars and operator semantics. Third, some studies on testing distributed deep learning systems involve training identical models under varying distributed configurations and verifying whether the prediction outputs remain equivalent within a predefined threshold [17]. Inspired by such an approach, we propose to construct the test oracle for GPU DBMSs based on the following assumption: within GPU DBMSs' heterogeneous architectures (which integrate both CPU and GPU devices), query execution should yield identical results regardless of whether the computation is performed on CPU-only or GPU-used executions. Our preliminary investigation (Section II-C) confirms the validity of this assumption. Thus, we propose to leverage such differential execution relation to construct a test oracle. Due to

Received 23 December 2024; revised 22 May 2025; accepted 24 May 2025. Date of publication 28 May 2025; date of current version 18 July 2025. This work was supported in part by the Natural Science Foundation of China under Grant 62272400. Recommended for acceptance by T. Zhang. *(Corresponding author: Rongxin Wu.)*

The authors are with the School of Informatics, Xiamen University, Xiamen 361102, China (e-mail: hujiaxin@stu.xmu.edu.cn; wurongxin@xmu.edu.cn).

Digital Object Identifier 10.1109/TSE.2025.3574328

0098-5589 © 2025 IEEE. All rights reserved, including rights for text and data mining, and training of artificial intelligence and similar technologies. Personal use is permitted, but republication/redistribution requires IEEE permission. See https://www.ieee.org/publications/rights/index.html for more information.

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

the aforementioned design of the test oracle, we need to address the second problem, i.e., how to generate query statements with configurations that can execute on the expected devices. Our preliminary investigation uncovered that existing GPU DBMSs provide configurations to facilitate the control of execution devices. However, unlike distributed configurations in deep learning systems—which are limited (e.g., only five key parameters) and exhibit deterministic behavior [17]—controlling CPU/GPU executions via configuration is non-trivial, as configurations may not consistently take effect across all query statements. Our preliminary study in *PG-Strom* showed that only 7.7% of randomly generated query statements, with GPU-related configurations turned on, are executed on GPU as expected. Thus, how to effectively generate query statements with appropriate configuration setups will greatly affect the effectiveness of GPU-related bug detection.

To address the aforementioned challenges, our basic idea is to leverage *association rule learning* [20], [21] to mine differential execution rules, which can characterize the underlying correlations among configurations, clauses and execution devices, from a small sample of randomly generated query statements with configurations. Firstly, we use a generator to randomly generate datasets and a labeler to distinguish between CPU-only and GPU-used query statements with configurations as labeled datasets. Then, we leverage an AST parser to extract configuration itemsets and clause sequences from both CPU-only and GPU-used SQL datasets. Next, we utilize an *association rule learning* algorithm to mine differential execution rules from configuration and clause lists. To increase the diversity of the generated query statements, we design an online interpreter, which synthesizes query statements with configurations based on differential execution rules with expression mutation. Finally, the detector hunts for GPU-related bugs by comparing pairs of returned query results.

We implemented our idea as a novel and general GPU DBMSs testing system named SQLaw[1]. To evaluate the performance of SQLaw, we selected three representative GPU DBMSs, either open source or commercial product with trial versions, including *PG-Strom*, *HeavyDB*, and *Dask-sql*. The evaluation results showed that SQLaw is effective in detecting GPU-related bugs. During the 24-hour run, SQLaw detected 51 previously unknown GPU-related bugs, while exesting state-of-the-art approaches could only detect 23 bugs. Upon submission, 51 out of 37 bugs were confirmed or fixed by project developers. Overall, we make the following contributions:

- To the best of our knowledge, we propose the first systematic study to detect GPU-related bugs in GPU DBMSs.
- We design an efficient offline execution rule learning approach to learn differential execution rules and an online interpreter to synthesize configurations and query statements with expected execution.
- We implement our idea as a DBMS testing system named SQLaw and systematically evaluate it using three representative GPU DBMSs. The results demonstrate the superiority of SQLaw in detecting GPU-related bugs.

[1]For SQLaw, *SQL* stands for *GPU DBMS*, and *Law* semantically echoes *Rule* in rule-based differential execution.

II. Background and Motivation

This section presents the background of our work, including the description of GPU DBMSs and typical GPU-related bugs. It also presents a preliminary study of detecting GPU-related bugs, and summarizes the findings about test oracle and the impact factors for determining execution devices in GPU DBMSs. Additionally, it discusses relevant challenges and motivations for this work.

A. GPU DBMSs

Concepts of GPU DBMSs. Unlike traditional DBMSs that only rely on Central Processing Units (CPUs) for data processing and query execution. GPU DBMS is a type of database management system that also utilizes Graphics Processing Units (GPUs) to accelerate database operations. As highly parallel processors, GPUs can significantly outperform CPUs, especially when handling large datasets and complex analyses [1], [2]. Consequently, there has been growing interest in utilizing GPUs to accelerate data analysis. Several GPU DBMSs have emerged in recent years, both in academic and industrial contexts [22], [23], [24], [25], [26], [27], [28], [29], [30], [31], [32].

GPU DBMSs Under Test. We select three mainstream GPU DBMSs as evaluation subjects in this work, including *PG-Strom* [29], *HeavyDB* [32], and *Dask-sql* [26]. We also seek to evaluate other GPU DBMSs. However, there are two problems that hinder us from testing them. One is that some GPU DBMSs are either closed source or commercial DBMSs that are not free for usage, such as *Brytlyt* [22], *Kinetica* [23], and *SQreamDB* [24]. The other is that some GPU DBMSs are open source but have been long unmaintained by developers (i.e., the developer of *BlazingSQL* has not maintained it for more than two years) such as *BlazingSQL* [25], *OmniSci* [30], and *MapD* [31]. We have also listed as many comparisons of currently known GPU DBMSs as possible in a public GitHub repository[2], in order to better justify our selection of the three GPU DBMSs under test.

Table I lists the DB-Engines Ranking[3] [33], Github stars, lines of code (LOC) and the first release year of GPU DBMSs under test. As *Dask-sql* is a recently released DBMS, it does not yet have a ranking. However, given that a large portion of major contributors are from NVIDIA Corporation, the world's premier GPU supplier, *Dask-sql* also has sufficient industry influence.

- **PG-Strom** [29] is an extension for *PostgreSQL*, a widely-used DBMS. It accelerates SQL workloads for analytics or batch processing on large datasets using CUDA as the execution backend, supporting offloading of operators like *SCAN*, *JOIN*, *GROUP BY*, etc. to the GPU device. It is self-adaptive, able to fall back to CPU execution if the query cannot run on the GPU device [34].

[2]https://github.com/qwebug/GPU-DBMS-list.git

[3]Since *PG-Strom* is an extension module of *PostgreSQL*, we use the ranking of *PostgreSQL* for *PG-Strom*.

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

TABLE I
DEMOGRAPHICS OF GPU DBMSS UNDER TEST

DBMS	Popularity Rank	Github Star	LOC	First Release
PG-Strom	4	1.3k	600k	2014
HeavyDB	156	3k	1727k	2017
Dask-sql	-	397	45k	2020

TABLE II
QUALITATIVE COMPARISON OF GPU DBMSS UNDER TEST

System	PG-Strom	HeavyDB	Dask-sql
Coverage	Scan Filter Group by Join Agg etc.	General Purpose	General Purpose
Backend	CUDA	LLVM to PTX	RAPIDS (cuDF)
Category	Traditional Database Extension	In-memory GPU Database	Non-memory GPU Database
Developer	HeteroDB, Inc	Heavy.AI	Dask
Open Source	Yes	Yes	Yes

- **HeavyDB** [32] is an integral part of the data analysis platform HEAVY.AI. It delivers high performance through rapid query compilation, query vectorization, and advanced memory management. Internally, it employs LLVM [35] to generate PTX code [36] for query execution. Furthermore, *HeavyDB* utilizes a customized LLVM pass to enable more adaptable and optimized execution strategies. Besides, *HeavyDB* stores all data in memory to fully leverage the performance potential of the GPU device and improve data processing speed [37].
- **Dask-sql** [26] is a distributed SQL query engine that allows querying and transforming data using a mix of common SQL operations and Python code. It supports running SQL queries on CUDA-enabled GPUs by utilizing RAPIDS [38] libraries like cuDF [39], to accelerate relational operations. Additionally, if users want to utilize the GPU for executing SQL, it is required to create GPU tables first in order to load data into GPU memory.

In summary, as shown in Table II, GPU DBMSs under test can be divided into three categories. Specifically, *PG-Strom*, *HeavyDB*, and *Dask-sql* are representative of the extension of traditional DBMSs, in-memory GPU DBMSs, and non-memory GPU DBMSs, respectively. The selected DBMSs are representative, encompassing three mainstream types of GPU DBMSs.

B. GPU-Related Bugs

Since all three GPU DBMSs under test are relational GPU DBMSs, we borrow the classification of bug types from traditional relational DBMSs to categorize bugs in GPU DBMSs into three types, and describe their features as follows.

- **GPU Crash Bugs** are software defects that can potentially lead to system instability, data corruption, or a complete shutdown of the DBMS. It is considered significant as they can disrupt normal database functions, potentially causing data loss or system downtime.

```
-- PG-Strom
SET pg_strom.enabled = OFF | ON;
SET pg_strom.enable_gpuscan = OFF | ON;
SET pg_strom.enable_gpuhashjoin= OFF | ON;
…  (10 configurations in total)
```

```
-- HeavyDB
ALTER SESSION SET EXECUTOR_DEVICE = 'CPU' | 'GPU';
ALTER SYSTEM CLEAR CPU | GPU | RENDER MEMORY;
-- Query Hints
SELECT /*+ cpu_mode */ …
SELECT /*+ overlaps_allow_gpu_build */ …
```

```
-- Dask-sql
CREATE TABLE table0 WITH(
              location = 'table0.csv', format = 'csv',
              persist = FALSE | TRUE, gpu = FALSE | TRUE);
```

Fig. 1. Configurations or hints about execution devices (pivotal configurations) in GPU DBMSs under test.

- **GPU Error Bugs** refer to software defects that result in error reports. These bugs appear as errors in GPU-accelerated database operations and may stem from problems in GPU code, GPU memory management, optimized GPU execution strategies, among other factors.
- **GPU Logic Bugs** are software defects that lead to incorrect behavior in the logical operations, data processing, or control flow of GPU programs. These bugs can result in unexpected and wrong outcomes in GPU calculations and data processing.

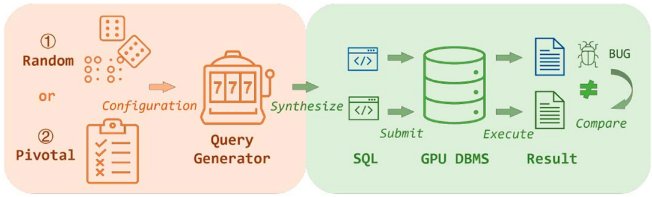
Unfortunately, despite the increasing popularity of GPU DBMSs and the potential for critical bugs, to the best of our knowledge, there has been no systematic effort to detect GPU-related bugs in GPU DBMSs.

C. Preliminary Study of Detecting GPU-Related Bugs

To detect GPU-related bugs, we have to address two fundamental problems. The first is to construct test oracle (i.e., determine whether a GPU DBMS behaves correctly for a given input). Our idea to resolve this problem is based on an observation that, the execution of query statements should return the same query results in both CPU and GPU devices. Therefore, we propose to leverage this differential execution relation to construct the test oracle for detecting GPU-related bugs. The second problem is to how to make the query statements executed on CPU and GPU devices. In our preliminary manual investigation in the technical documents, we find 10, 4 and 2 configurations or hints about execution devices in *PG-Strom* [34], *HeavyDB* [37], and *Dask-sql* [40] respectively, as shown in Fig. 1. To ease the explanation, we refer to these configurations as the pivotal configurations in the following description. To control the execution devices, we leverage the configurations of each GPU DBMS in two approaches. The first approach is to randomly enable one or more configurations (not only pivotal configurations) for each query statement. The second is to enable pivotal configurations which are discovered by our manual investigation in the technical documents of GPU DBMSs and then set up these configuration options for both CPU-only and GPU-used execution.

Our test oracle is designed under the assumption that query statements should yield identical results on both CPU and GPU

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.



(a) Generator (b) Bug Detector

Fig. 2. Workflow of differential execution with random or pivotal configurations.

devices. To validate this, we randomly generated 1,000 pairs of query statements under pivotal configurations—covering diverse query types and data sizes used in our experiments—for each of the three GPU DBMSs under evaluation. By executing each query pair and comparing their results, our preliminary study confirms this assumption: the outputs from CPU-only and GPU-enabled executions are consistent, except in cases where bugs are present. Leveraging this test oracle, we developed a preliminary differential execution framework (see Fig. 2) to detect GPU-related bugs. In a 3-hour testing period, the framework with random configurations uncovered 5, 3, and 2 unique GPU-related bugs in *PG-Strom*, *HeavyDB*, and *Dask-sql*, respectively, while the framework with the pivotal configurations detected 7, 4, and 3 bugs.

Finding 1: The differential execution relation is a test oracle that can be used to detect GPU-related bugs.

Upon statistics on *PG-Strom*, we observe that only 3.4% of query statements with random configurations and 7.7% of query statements with the specified pivotal configurations are executed on GPU devices as expected. Because the pivotal configurations can better control query statements execution on expected devices than the random configurations, the differential execution framework with the pivotal configurations detects more GPU-related bugs than the one with random configurations within the same time budget.

Finding 2: Effective control of query statements executed on expected devices is key to improving the efficiency of GPU-related bug detection.

We further investigate the bugs detected by the random configurations and the pivotal configurations respectively. It is interesting to find that there are still some bugs detected by the random configurations while missed by the pivotal configurations, even though the same query statements are executed. For example, in *Heavydb*, with the pivotal configurations both *EXECUTOR DEVICE* and *ALTER SYSTEM CLEAR* set to *'CPU'* or *'GPU'*, the query statement *SELECT * FROM t0 LIMIT 1 OFFSET 1;* results in the same output. However, with the random configurations, we discovered that adding the query hint */*+ keep_result */*, which is essentially one kind of configuration but not the a pivotal one for execution devices, results in

```
1  -- Query with Configurations Expected for CPU-only Execution
2  -- Example 1  PG-Strom  -->  GPU-used Execution
3  SET pg_strom.enable_gpupreagg = OFF;
4  SELECT t0.c0 FROM t0 NATURAL JOIN t1;

5  -- Query with Configurations Expected for GPU-used Execution
6  -- Example 2  PG-Strom  -->  CPU-only Execution
7  SET pg_strom.enabled = ON;
8  SET pg_strom.enable_gpuscan = ON;
9  …  -- Set All of Configurations ON
10 SELECT c0 FROM t0;
11 -- Example 3  HeavyDB  -->  CPU-only Execution
12 ALTER SESSION SET EXECUTOR_DEVICE = 'GPU';
13 SELECT /*+ cpu_mode */ t0.c0 FROM t0 WHERE t0.c1>0;

14 -- Query with mixed CPU and GPU tables
15 -- Example 4  Dask-sql  -->  Error Execution
16 CREATE TABLE table0/ table1_gpu/… WITH( …, gpu=FALSE|TRUE);…
17 SELECT table0.c0 FROM table0, table1_gpu, table2;
```

Fig. 3. Challenge examples of unexpected execution.

different outputs, uncovering a GPU-related bug. The missing search of other configurations makes the framework with the pivotal configurations fail to discover some GPU-related bugs.

Finding 3: Compared to the approach with random configurations, the approach with the pivotal configurations can benefit more efficient GPU-related bug detection, while they would miss some GPU-related bugs due to the incomplete configuration search space.

D. Challenge

Finding 2 and 3 indicate that how to generate SQL statements and the corresponding configurations to enable CPU/GPU execution is the foundation of detecting GPU-related bugs, affecting both the efficiency and the effectiveness. Neither the random configuration approach nor the approach with the pivotal configurations is the perfect solution. Besides, existing GPU-related differential testing approaches leverage the pivotal configurations to deterministically control the execution environments, such as distribution settings [17]. Similarly, we exploit pivotal configurations detailed in § II-C to control execution devices in GPU DBMSs. As shown in Fig. 5, *Motivating Example 1* presents a pair of generated query statements with the pivotal configurations in *PG-Strom*. Despite setting all pivotal configurations for GPU-used execution, the query statement in *Line 7* is still executed on CPU devices. Since both query statements (*Line 4* and *Line 7*) are executed on CPU devices, this comparison is considered invalid according to our proposed test oracle. By investigating the generated statements and the corresponding configurations in our preliminary study, we find it non-trivial to make the query statements executed on the expected devices through configurations, since some configurations would not always take effect in every statement. Fig. 3 gives four classical scenarios for the executions in the unexpected devices.

The first is that some configurations limit their effect in certain clause patterns. For instance, in *Example 1*, the GPU-related configuration *enable_gpupreagg* will only take effect in the statements with aggregate functions. Since the statement at *Line 4* does not match such pattern, it will

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

not enable GPU execution. The second is that some clause patterns cannot benefit from GPU acceleration and thus none of the GPU-related configurations can enable their GPU-used execution. *Example 2* shows a case where the statements at *Line 10* will always get executed on CPU devices even when turning on all of GPU configurations. The third is that the priority of configurations differs in the scope of clauses. For instance, the query hint `/*+ cpu_mode */` in *Example 3* has a higher priority than the global configuration `EXECUTOR_DEVICE` in the statement at *Line 12*, and thus it will force the statement at *Line 13* to get executed on CPU devices. The fourth is that a query statement cannot use both CPU and GPU tables simultaneously. *Example 4* presents a case where the query statement with mixed CPU and GPU tables after *FROM* clause at *Line 17* leads to error execution.

E. Key Idea

To effectively and efficiently detect GPU-related bugs, we need to not only generate query statements but also generate the appropriate configurations to control their execution in both CPU and GPU devices. Essentially, the key to address this problem is to uncover the correlation among query statements, configurations and the execution devices.

Inspired by the prior studies of rule learning [20], which aim to extract rules, relationships and patterns from data [41], [42], we propose to leverage *association rule learning* to extract the patterns that characterize the relation between query statements with configuration setups and execution devices.

To facilitate the understanding on how our association rule can assist the generation of query statements with configuration settings, we give several examples as shown in Fig. 4. With a small sampling from the randomly generated statements with configurations, we can perform the statistics about the configuration itemset and clause sequences. We then divide the statistics into two tables based on the execution devices. For each item in the table, we leverage two pivotal metrics, *support* and *confidence* (see §III-B4), to describe how much probability a clause sequence will be executed on the GPU device and a configuration itemset can control expected execution. For *Example 1* in Fig. 3, it matches the pattern of *Row 4* in Fig. 4, and *confidence* value is larger than *GPU-used configuration threshold* which indicates that it is highly likely to be executed on the GPU device. For *Example 2* in Fig. 3, it matches the pattern of *Row 1* in Fig. 4, *support* value is 0.00 (less than *GPU-used clause threshold*) which indicates that such clause sequence will always be executed on the CPU device. For *Example 3* in Fig. 3, it matches the patterns of *Row 3* in Fig. 4, and *confidence* value is larger than *CPU-only configuration threshold* which suggests that it is high likelihood of CPU-only execution. By merging the two statistical tables about CPU and GPU execution, we can get the table of differential execution rules, which is used to characterize how to generate the configurations and query statements so as to make generated query statements to be executed on both CPU and GPU devices.

In Fig. 5, *Motivating Example 2* illustrates a pair of query statements generated using differential execution rules (*Lines 9-10*) in *PG-Strom*. Since two query statements (*Line 12* and *Line 14*) are executed as expected on both CPU and GPU devices, the comparison between the return results of both statements satisfies our proposed test oracle. Importantly, this comparison reveals a GPU-related bug due to inconsistent execution results. In our experiments (see § V-E), the statistics from *PG-Strom* reveal that only 7.7% of queries with configurations similar to *Motivating Example 1* execute on the expected devices. However, when leveraging differential execution rules, this success rate rises to 40.1%, a 4.2× improvement. This enhancement strengthens the test oracle's reliability and significantly aids in detecting GPU-related bugs.

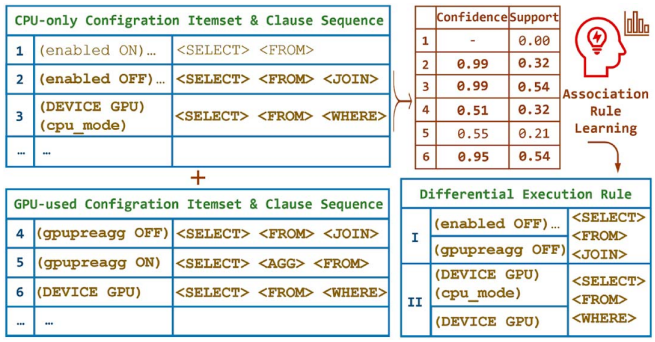


Fig. 4. Key idea of *association rule learning*.

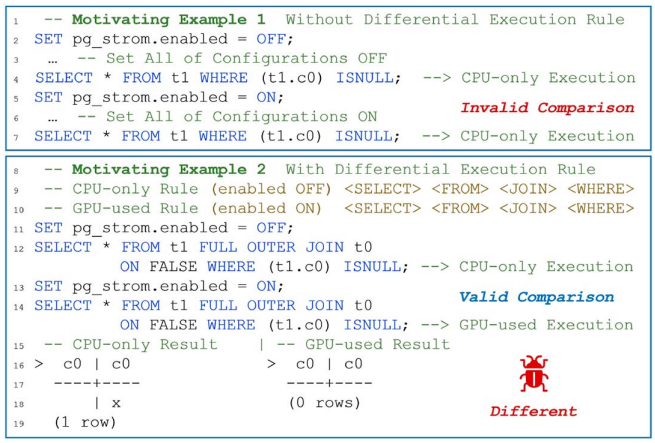


Fig. 5. Motivating examples of differential execution rules to assist in GPU-related bug detection.

III. Approach

As described in § II, our goal is to improve our preliminary study to generate expected query statements with configurations for more efficient and general detection of GPU-related bugs. In this section, we present our design of SQLaw. Specifically, we discuss how to combine offline rule learning and online interpreter with mutation for testing.

A. Overview of SQLaw

Fig. 6 presents the workflow of SQLaw, a rule-based differential execution framework, integrating *Offline Rule Learning*, *Online Interpreter with Mutation*, and *Bug Detector*.

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

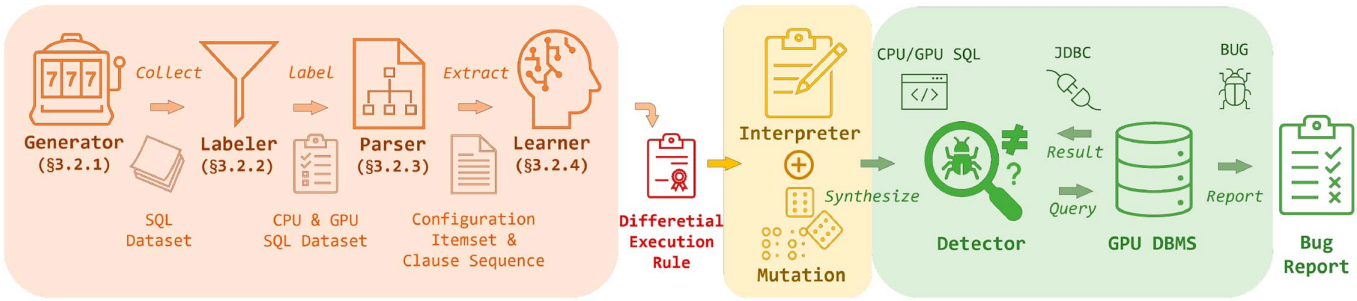


Fig. 6. Workflow of SQLaw. *Generator* initially generates the SQL dataset as input for offline rule learning. After labeling and parsing, *Learner* mines differential execution rules from configuration itemsets and clause sequences. These rules then act as input for the online interpreter with mutation, aiding in the generation of query statements with configurations executing on expected CPU/GPU devices. Finally, *Bug Detector* submits CPU/GPU queries to the GPU DBMS and compares their results. If an inconsistency is detected, a GPU-related bug report is generated as output.

Step 1: We first use *Generator* (§III-B1) to randomly generate a sample of configurations and query statements. After verifying their correctness, we collect them into an SQL dataset. Then, we utilize *Labeler* (§III-B2) to label CPU-only and GPU-used query statements with configurations as CPU-only and GPU-used SQL dataset. Next, we extract configuration and clause itemsets from CPU-only and GPU-used SQL datasets using *Parser* (§III-B3). Finally, we learn the underlying correlations (regarded as differential execution rules) between execution devices and query statements with configurations from configuration itemsets and clause sequences through *association rule learning* (§III-B4).

Step 2: We employ *Interpreter with Mutation* (§III-C) to mutate expressions and synthesize configurations and query statements based on differential execution rules.

Step 3: We leverage *Bug Detector* (§III-D) to submit pairs of CPU-only and GPU-used query statements with configurations to GPU DBMSs for execution, and then compare their query results for detecting GPU-related bugs.

B. Offline Rule Learning

To learn differential execution rules, we propose a workflow consisting of four components: *Generator*, *Labeler*, *Parser* and *Learner* for offline rule learning.

1) Generator: To obtain a SQL dataset for *association rule learning*, we implement an AST-based generator to randomly generate a set of query statements with configurations. As shown in Fig. 7, the generation process comprises two major parts as follows.

On one hand, *Generator* randomly selects one or more configuration nodes from configuration lists and matches them with random options as configuration statements.

On the other hand, to generate query statements randomly, *Generator* initially builds an Abstract Syntax Tree (AST) for both the clause and expression layers based on syntax rules. Then it executes random mutations and modifications on clause and expression nodes within the AST. This may involve operations like inserting, deleting, and replacing nodes in the AST, as well as changing node attributes. Furthermore, to enhance the diversity of generated query statements, we randomize selection

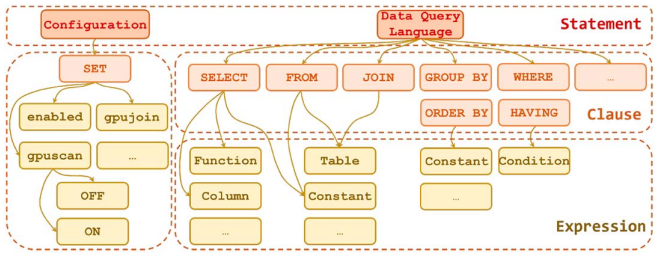


Fig. 7. Detailed workflow of an AST-based *SELECT* query statement with *SET* configuration generator.

of tables, columns, and operators, and introduce random logical conditions in *WHERE* and *HAVING* clauses.

Finally, generated configuration statements and a query statement compose a group of query statements with configurations. *Generator* then verifies the syntactic and semantic correctness of generated query statements with configurations by executing them in databases. Groups of verified query statements with configurations form an SQL dataset.

2) Labeler: To label the SQL dataset from the generator, we design a labeler to mark generated query statements with configurations as either CPU-only or GPU-used.

It should be noted that all of the DBMSs under test can report query execution plans, while only the query execution plans of *PG-Strom* and *HeavyDB* indicate whether GPU execution and optimization are enabled. *Dask-sql*'s query execution plans do not provide information about GPU execution. To address this, we propose using program execution traces to confirm whether query statements invoke GPU-related libraries during execution. If GPU-related libraries are invoked, it proves that these statements executed on the GPU device.

- **Query Execution Plan**. The query execution plan is a list of instructions that describes how DBMSs execute a query statement [43]. Both *PG-Strom* and *HeavyDB* can use *EXPLAIN* command to check whether a query statement is executed on the GPU device or not. If keywords such as GpuPreAgg, GPU Projection, IR for the GPU, and so on appear in the output log, it indicates that the query statement was executed on the GPU device [34], [37]. Fig. 8 shows two examples of *EXPLAIN* command

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

```
-- PG-Strom
SET pg_strom.enabled = ON;
EXPLAIN SELECT COUNT(*) FROM table0;

                        QUERY PLAN
-----------------------------------------------------------
     Aggregate (cost=146.81..146.82 rows=1 width=8)
      -> Custom Scan (GpuPreAgg) on table0
            (cost=146.79..146.80 rows=1 width=8)
         GPU Projection: pgstrom.nrows()
      (3 rows)
```

```
-- HeavyDB
ALTER SESSION SET EXECUTOR_DEVICE = 'GPU';
EXPLAIN SELECT /*+ allow_loop_join */ COUNT(*) FROM table0;

Explanation
IR for the GPU:
===============
Function Attrs: mustprogress nounwind uwtable
define dso_local void @multifrag_query_hoisted_literals
    (i32* noundef %0, , i32* noundef %1, i64** noundef %2, …
 …
```

Fig. 8. Examples of query execution plans in *PG-Strom* and *HeavyDB*, respectively.

```
# trace_sql.py
import trace
 …
def exec_sql():
   c.sql("SELECT COUNT(*) FROM table0").compute()
 …
tracer = trace.Trace(count=False, trace=True)
tracer.runfunc(exec_sql)

# python trace_sql.py
> === modulename: trace_sql, funcname: exec_sql
 trace_sql.py(7): c.sql("SELECT COUNT(*) FROM table0").compute()
 …
 types.py(133):    cudf.Scalar,
 types.py(134):    cudf.lib.scalar.DeviceScalar,
 types.py(135):    cudf.core.tools.datatimes.DataOffset,
 …
```

Fig. 9. An example of program execution trace in *Dask-sql*.

output log for query statements that enable GPU execution in *PG-Strom* and *HeavyDB*, respectively.

- **Program Execution Trace**. The trace module in Python can be used to trace program execution, print caller or callee relationships and list executed functions during a program run [44]. As shown in Fig. 9, GPU-related libraries invoked are printed, such as *cuDF* [39], if a query statement is executed on the GPU device during tracing.

3) Parser: To pre-process and extract key features from CPU-only and GPU-used SQL datasets, we leverage an AST parser to distill the configuration and clause structure of each group of query statements with configurations. We implemented such a parser with the help of the open-source *ANTLR4* [45]. *ANTLR* (ANother Tool for Language Recognition) is a parser generator that uses a LL(*) algorithm for parsing [46], in computer based language recognition.

The first step involves specifying syntax rules to define SQL syntax using a custom syntax definition language. This includes defining the structure of query statements with configurations, such as *SET* clauses, *SELECT* clauses, *FROM* clauses, *WHERE* clauses, and others. Then *ANTLR* processes the defined syntax rules to generate corresponding parser code including lexical analyzer and parser classes to parse query statements with configurations.

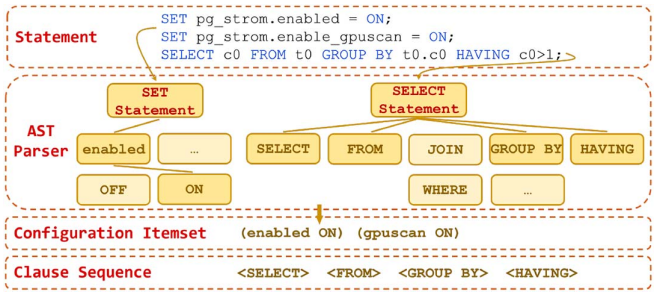


Fig. 10. Detailed workflow of *SET* configuration and *SELECT* query statement parser.

Fig. 10 illustrates the process of parsing a group of query statements with configurations, these with *SET* configuration statement and *SELECT* query statement. The AST parser extracts a series of configuration itemsets and clause sequences from these statements separately according to the syntax rule definition file. It then takes these configuration itemsets and clause sequences into a configuration list and a clause list in sequential order, respectively, as training sets.

4) Learner: The goal of *Learner* is to mine differential execution rules from training sets to improve the generation of query statements with configurations for expected execution. Essentially, *association rule learning* is used to learn how frequently items of a set appear together. We adapt some of the definitions in *association rule learning* [20], [21] to fit our scenario. In our work, we aim to learn how frequently both configuration itemsets and clause sequences of query statements with configurations are associated with execution devices in practice. There are several key definitions as follows.

Definition 3.1: A pair of differential execution rules is defined as r,

$$r = \begin{cases} r_{fc} \wedge r_l & \text{(CPU-only execution)} \\ r_{fg} \wedge r_l & \text{(GPU-used execution)} \end{cases}$$

where r_l, r_{fc} and r_{fg} denote a clause rule, a configuration rule on CPU-only execution and a configuration rule on GPU-used execution, respectively.

To construct a pair of differential execution rules from training sets, we utilize two pivotal metrics, $support$ and $confidence$, to evaluate how much probability a clause rule supports GPU-used execution excluding the effects of configuration rules and how much probability a configuration rule influences the same clause rule on execution devices, respectively.

Definition 3.2: Let

$$support(s) = \frac{|S_g|}{|S|}$$

represent $support$ for s, meaning how frequently a clause sequence s in a list of total clause sequences S belongs to GPU-used execution. Besides, S_g is a list of GPU-used clause sequences and $|\cdot|$ denotes the number of elements in the list. If $support(s) > t_l$, s is recognized as a clause rule r_l, where t_l is the threshold of clause rules.

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

Algorithm 1: Learning Differential Execution Rule

Input : $\{S\}$: clause sequence list, $\{I\}$: configuration itemset list, t_l, t_{fc}, t_{fg}: threshold values
Output: $\{r\}$: differential execution rule list

Function *Support*(N, M):
  return *listSize*(N) / *listSize*(M);
Function *Confidence*(N, M):
  return *listSize*(N) / *listSize*(M);
foreach *different* s *in GPU-used clause sequence list* **do**
  $S \leftarrow$ *all clause sequence list for* s;
  $S_c \leftarrow$ *CPU-only clause sequence list for* s;
  $S_g \leftarrow$ *GPU-used clause sequence list for* s;
  // learning clause rule
  if *Support*(S_g, S) $> t_l$ **then**
    $r_l \leftarrow s$;
    $I_c \leftarrow$ *CPU-only configuration itemset list for* r_l;
    $I_g \leftarrow$ *GPU-used configuration itemset list for* r_l;
    // learning configuration rule
    if *Confidence*(I_c, S_c) $> t_{fc}$ **then**
      $r_{fc} \leftarrow i_c$; // CPU-only configuration rule
    if *Confidence*(I_g, S_g) $> t_{fg}$ **then**
      $r_{fg} \leftarrow i_g$; // GPU-used configuration rule
    // a pair of differential execution rule
    $r = r_{fc} \wedge r_l, r_{fg} \wedge r_l$;
    add r *to* $\{r\}$;
return $\{r\}$;

Definition 3.3: Given the same clause rule r_l, let

$$confidence(r_f \rightarrow i_c) = \frac{support(r_f \rightarrow i_c)}{support(r_f)} = \frac{|I_c|}{|S_c|}$$

represent $confidence$ of r_f inferring i_c, meaning how much probability a configuration itemset i_c from the CPU-only configuration list leads the same clause rule r_l to CPU-only execution, where I_c and S_c denote a list of i_c belonging to r_l in the CPU-only configuration list and a list of r_l in the CPU-only clause list, respectively. If $confidence(r_f \rightarrow i_c) > t_{fc}$, i_c is recognized as a CPU-only configuration rule r_{fc}, where t_{fc} is the threshold of CPU-only configuration rules.

Equally, let

$$confidence(r_f \rightarrow i_g) = \frac{support(r_f \rightarrow i_g)}{support(r_f)} = \frac{|I_g|}{|S_g|}$$

represent $confidence$ of r_f inferring i_g. If $confidence(r_f \rightarrow i_g) > t_{fg}$, i_g is recognized as a GPU-used configuration rule r_{fg}, where t_{fg} is the threshold of GPU-used configuration rules.

Definition 3.4: Let

$$Learner(\mathcal{D}) = \{r | support(r_l) > t_l \wedge confidence(r_{fc}) > t_{fc} \wedge confidence(r_{fg}) > t_{fg}\}$$

represent $Learner$ learns the differential execution rule list $\{r\}$ from training sets $\mathcal{D}$, according to $support$ and $confidence$. Algorithm 1 shows the detailed process for learning differential execution rules.

C. Online Interpreter With Mutation

The SQL dataset contains only a small sample of random and unpaired query statements with configurations. Due to the limited number of query statements with configurations in the SQL dataset, we are unable to fully test GPU DBMSs. Therefore, we propose to generate more diverse query statements with configurations by mutating expressions.

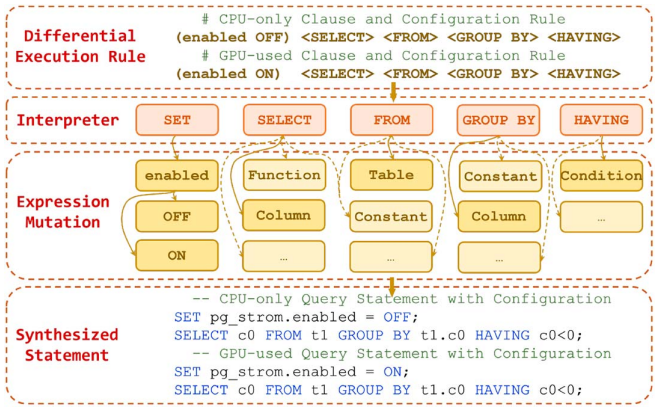


Fig. 11. Detailed workflow of the interpreter with expression mutation.

In concrete terms, we design an online interpreter with expression mutation that can randomly mutate expressions and synthesize configurations and query statements according to differential execution rules, as shown in Fig. 11.

First, a pair of differential execution rules is randomly selected from the rule list and submitted to the interpreter. In *Differential Execution Rule* part of Fig. 11, there is a selected pair of differential execution rules, including a clause rule and a pair of CPU-only and GPU-used configuration rules.

Next, *Interpreter* generates configuration statements according to configuration rules, synthesizes query statements based on the clause rule and mutates expressions. On one hand, *Interpreter* synthesizes a pair of complete CPU-only and GPU-used configuration statements based on the corresponding syntax rules of the database dialect and the given configuration rules. On the other hand, *Interpreter* randomly mutates corresponding expressions for each clause item following the given clause rule. As shown in *Interpreter and Expression Mutation* parts of Fig. 11, *Interpreter* first constructs configuration statements for differential execution based on the configuration rules, then constructs the syntactic structure of the query statement based on the clause rule and mutates a randomly generated expression after the clause, e.g., *Column*, *Table*, *Condition*, etc.

Finally, *Interpreter* matches a pair of differential configurations with the corresponding query statement. In *Synthesized Statement* part of Fig. 11, these synthesized statements are a typical group of differential configurations and query statements according to differential execution rules.

D. Bug Detector

Building on the test oracle of differential execution on both CPU and GPU devices, we implement a bug detector with a detailed workflow illustrated in Fig. 12. To minimize intentionally different query results, we align the settings for the GPU query regarding numerical precision and optimization with those for the CPU query before testing, following the technical manuals.

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

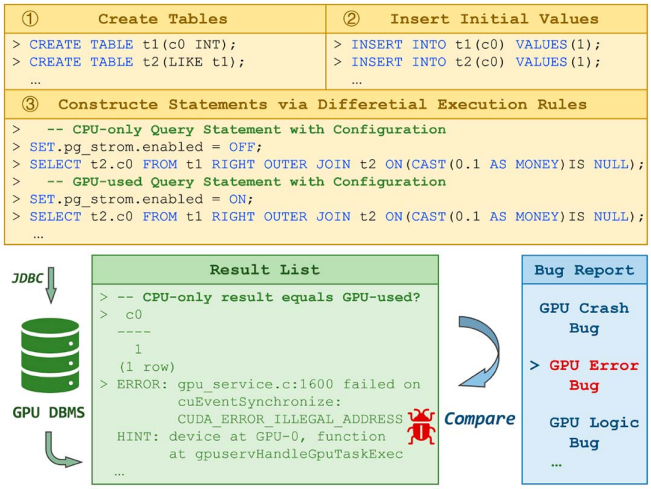


Fig. 12. Detailed workflow of the bug detector.

The bug detector first creates tables and inserts initial values for data preparation. It then gets pairs of CPU-only and GPU-used query statements with configurations synthesized by *Interpreter* according to differential execution rules.

Next, the bug detector submits CPU-only query statements with configurations to GPU DBMSs. If these statements lead to errors, it skips to the next pair, effectively filtering out non-GPU bugs. And if these statements normally return results, it submits GPU-used query statements with configurations and compares the two returned results of CPU-only and GPU-used ones. A bug is detected if there is a difference in the two results, or if a crash or error is reported. In Fig. 12, it compares the results of CPU-only and GPU-used configurations and query statements returned from GPU DBMSs and identifies a GPU error bug.

Finally, the bug detector summarizes identified GPU-related bugs, removes duplicate instances, and generates a meticulously detailed bug report.

E. Detailed Process of SQLaw

To provide more technical details, we describe the process of SQLaw using *PG-Strom* as an example in Fig. 13.

Step 1: To collect datasets, *Generator* randomly generates a certain number of query statements with configurations, constituting a SQL dataset, after verifying their correctness.

Step 2: To label query statements with configurations in the collected dataset, *Labeler* leverages the query execution plan approach to check whether each query statement with configurations is executed on the CPU or GPU device. These labeled query statements with configurations are then categorized into the CPU-only and GPU-used SQL datasets.

Step 3: To pre-process datasets, *Parser* extracts configuration itemsets and clause sequences from the CPU-only and GPU-used SQL datasets respectively. For example, the configuration statement *SET pg_strom.enabled = ON;* is parsed as *(enabled ON)* and the query statement *SELECT c0 FROM t0;* is parsed as *< SELECT> <FROM>* .

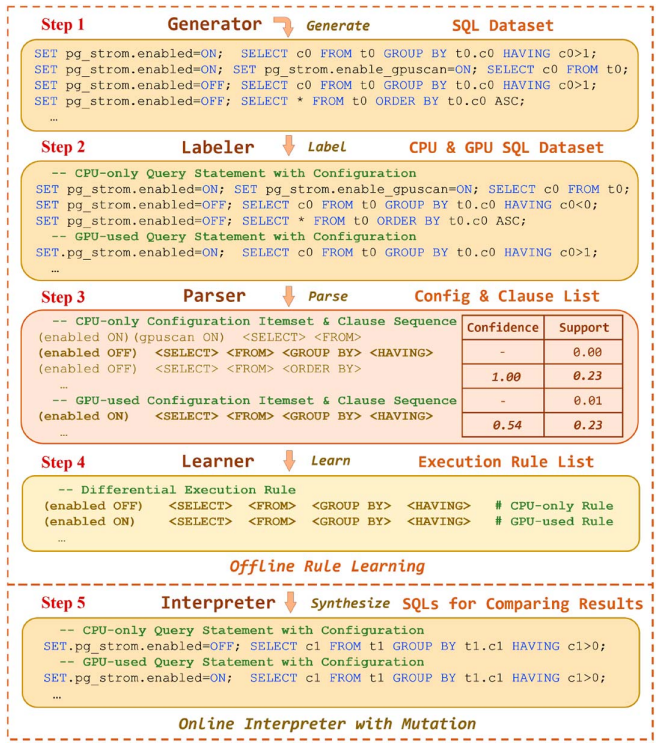


Fig. 13. Detailed process of combining offline rule learning and online interpreter with mutation for bug detection.

Step 4: *Learner* mines a pair of differential execution rules from configuration and clause itemsets by calculating $support$ and $confidence$. It first uses $support$ to select clause rules, for example, the clause sequence *< SELECT> <FROM> <GROUP BY> <HAVING>* is selected as a clause rule because its $support$ value is 0.23 and higher than the threshold value. Then, it utilizes $confidence$ to match a pair of CPU-only and GPU-used configuration rules for this clause rule. For example, *(enabled OFF)* and *(enabled ON)* are regarded as CPU-only and GPU-used configuration rules respectively, because their $confidence$ values (1.00 and 0.54) are both higher than the threshold value.

Step 5: *Interpreter with Mutation* mutates expressions and synthesizes a pair of query statements with configurations based on the differential execution rules.

Finally, *Bug Detector* submits this pair of CPU-only and GPU-used query statements with configurations respectively, and compare their query results returned by the GPU DBMS for GPU-related bugs detection.

IV. Implementation

We have implemented our approach SQLaw as a GPU-related bug detection tool, written in Java. The implementation process involves two steps: offline rule learning for differential execution rules and online application of these rules for bug detection. We present some implementation details below.

As *PG-Strom* has a lower percentage of randomly generated query statements with configurations capable of GPU-used execution than the other two databases, *Generator* needs

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

TABLE III
VERSION INFORMATION OF THREE GPU DBMSS

DBMS	PG-Strom	HeavyDB	Dask-sql
Version	Commit 9765660 in Github (Based on PostgreSQL 15.6)	7.1.0-20230821 -eae9ec17da	2023.6.0

to generate more to ensure enough GPU-used statements for *Learner*. Thus, we leveraged *Generator* to generate 1 million groups of query statements with configurations for *PG-Strom*, and 100 thousand groups for *HeavyDB* and *Dask-sql*.

Parser is constructed based on *ANTLR4* version 4.12.0, and the threshold values t_l, t_{fc} and t_{fg} in *Learner* are set to 0.1, 0.9 and 0.5, respectively. The justification of the threshold selection will be discussed in §V-E. Finally, *Learner* mined 2,082, 1,378 and 617 pairs of differential execution rules from *PG-Strom*, *HeavyDB* and *Dask-sql*, respectively.

After bug detection implementation, we obtained numerous duplicate bug reports. To identify unique GPU-related bugs, we imitated the statement-level and syntax-based reduction in *SQLancer* [47] to automatically simplify bug reports. Then, for better distinction, we also borrowed ideas from *delta debugging* [48] to localize the root cause of simplified bug reports. If two bug reports belong to the same root cause, we will consider them duplicated.

V. EVALUATION

After presenting the design of SQLaw, we evaluate its effectiveness and efficiency in detecting GPU-related bugs in real-world GPU DBMSs. Our evaluation aims to address the following research questions:

- **RQ1:** How many GPU-related bugs in real-world GPU DBMSs can be detected by SQLaw?
- **RQ2:** Can SQLaw outperform existing state-of-the-art bug detection approaches?
- **RQ3:** How do differential execution rules in SQLaw benefit GPU-related bug detection?

A. Evaluation Step

Environment. Experiments were conducted on a GPU server equipped with an 8-core Intel Core i7-11700 CPU@2.50GHz, 64GB DDR4 memory, 1TB SSD, a single NVIDIA GeForce RTX 4090 (24GB) GPU [49] and a single NVIDIA Tesla V100 (16GB) GPU [50] (only used to evaluate the impact of GPU hardware devices in §V-E) running Driver 535.171.04 and CUDA 12.2. The server ran Ubuntu 24.04 LTS operating system with 6.8.0-31-generic Linux kernel. We deployed three GPU DBMSs for testing via Docker 26.1.1.

GPU DBMSs Under Test. We selected three mainstream GPU DBMSs as evaluation subjects, including *PG-Strom* [29], *HeavyDB* [32], and *Dask-sql* [26], as described in Table I. We chose the latest version of each GPU DBMS at the time of conducting our experiments. The version information is listed in Table III.

Baselines and Ablations. Since SQLaw is the first tool for detecting bugs in GPU DBMSs, we can only adapt existing state-of-the-art approaches developed for non-GPU DBMSs as the baselines. We selected three representative state-of-the-art approaches, including *SQLsmith*, *NoREC* and *TLP*. In particular, *SQLsmith* is one of the most effective approaches for detecting crash and error bugs, while both *NoREC* and *TLP* represent advanced approaches for detecting logic bugs. Each of them has found a total of over a hundred previously unknown bugs in various non-GPU DBMSs [51], [52], [53]. We briefly introduce the methodologies of each baseline as follows.

- **SQLsmith** [51] generates random query statements using an AST model and checks whether the DBMS works normally when executing them.
- **NoREC** [52] translates query statements into non-optimized queries in the optimizer and compares the results to identify logic bugs.
- **TLP** [53] partitions an original query into three separated queries by decomposing its predicate, and triggers a logic bug when the union result of these separated queries mismatches the original one.

Additionally, we adapted the above three approaches to detect GPU-related bugs through two key modifications. First, we extended these three approaches to support the dialects of the three GPU DBMSs under test. Second, we introduced a configuration step (following the pivotal configuration approach in § II-C) to control query statements on GPU-used execution. Specifically, this step involves setting all pivotal configuration options shown in Fig. 1 for GPU-used execution before executing query statements. This increases the percentage of generated query statements executed on GPU devices compared to executions without configuration. Consequently, we refer to the adapted approaches as *SQLsmith**, *NoREC** and *TLP**.

Since SQLaw represents a novel extension of differential testing, we compared its performance against a conventional differential testing approach in the ablation study (§ V-D). This differential testing approach utilizes *Generator* in §III-B1 to generate query statements, adopts the pivotal configuration approach detailed in § II-C to control CPU-only and GPU-used execution, and compares their results to identify bugs.

B. Effectiveness and Efficiency of SQLaw

To evaluate the effectiveness and efficiency of SQLaw, we used it to detect GPU-related bugs in *PG-Strom*, *HeavyDB*, and *Dask-sql*. As shown in Table IV, the column **Detected** presents the number of unique bugs detected in three GPU DBMSs by SQLaw. SQLaw successfully detected a total of 51 bugs, including 22 in *PG-Strom*, 15 in *HeavyDB*, and 14 in *Dask-sql*. The column **Confirmed** presents the number of these unique bugs confirmed by the developer on Github. Developers of three GPU DBMSs have confirmed 37 bugs in total, with a confirmed ratio of up to 73%. This result indicates that SQLaw is able to effectively detect real-world GPU-related bugs. The column **Fixed** presents the number of these unique bugs the developer has already fixed. Thanks to developers' efforts, a total of 36 bugs have been fixed, with a fixed ratio of up to 71%, including 22 in *PG-Strom*, 10 in *HeavyDB*, and 5 in *Dask-sql*.

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

TABLE IV
UNIQUE BUGS REPORTS OF THREE GPU DBMSS

DBMS	Detected	Confirmed	Fixed
PG-Strom	22	22	22
HeavyDB	15	10	9
Dask-sql	14	5	5
Total	51	37	36
Proportion	-	73% (37/51)	71% (36/51)

TABLE V
THE STATISTICS OF THE THREE TYPES OF BUGS

DBMS	GPU Crash Bug	GPU Error Bug	GPU Logic Bug
PG-Strom	1	10	11
HeavyDB	3	3	9
Dask-sql	0	7	7
Total	4	20	27

The developers of *PG-Strom* submitted the fix commits directly to the GitHub repository, while the developers of *HeavyDB* and *Dask-sql* fixed these bugs on new releases through internal tickets. To keep track of the status of our reported bugs, we release the bug list in a public GitHub repository[4].

Classification of Detected GPU-related Bugs. As described in §II-B, GPU-related bugs can be classified into three types, including GPU crash bugs, GPU error bugs, and GPU logic bugs. Overall, as shown in Table V, SQLaw can effectively cover all three types of GPU-related bugs in three GPU DBMSs under test. Regarding GPU crash bugs, they are severe and may cause complete shutdown of GPU DBMSs. SQLaw is capable of effectively detecting 1 and 3 bugs in *PG-Strom* and *HeavyDB*, respectively. For GPU error bugs, they appear in GPU-accelerated database operations. SQLaw is effective in detecting 10, 3, and 7 bugs in these GPU DBMSs, respectively. GPU logic bugs are the most insidious among the three bug types. Despite this, SQLaw still effectively detects 11, 9, and 7 bugs in these GPU DBMSs, respectively.

To better understand the landscape of trigger patterns of GPU-related bugs, we further analyzed their query statements with configurations and extracted trigger patterns. As shown in Table VI, these trigger patterns fall into three categories. The first category is the pattern related to clause and aggregate function. This is the primary trigger pattern category, accounting for 60.3% of the bugs. The second category is the pattern related to expression, value, and table operation. This is another major trigger pattern category, accounting for 34.2%. The last category is the pattern related to configuration and hint, accounting for 5.5%. In addition, these three categories of trigger patterns correspond to clause rules, configuration rules and the expression mutation. This suggests that differential execution rules, mined through offline rule learning in *Learner*, and the mutation in *Interpreter* aid in triggering GPU-related bugs.

[4]https://github.com/qwebug/bug-report.git

TABLE VI
THE LANDSCAPE OF TRIGGER PATTERNS FOR GPU-RELATED BUGS (A GPU-RELATED BUG MAY CONSIST OF ONE OR MORE TRIGGER PATTERNS)

Trigger Pattern	PG-Strom	HeavyDB	Dask-sql	Sum
SELECT Clause	1	1	4	6
DISTINCT Clause	0	1	0	1
FROM Clause	0	0	1	1
WHERE Clause	5	3	0	8
JOIN Clause	9	2	1	12
GROUP BY Clause	3	0	0	3
HAVING Clause	1	0	2	3
ORDER BY Clause	0	1	0	1
OFFSET Clause	0	2	0	2
LIMIT Clause	0	1	0	1
Aggregate Function	5	1	0	6
Special Expression	2	0	6	8
Type CAST	3	3	1	7
Operator	0	2	2	4
Special Value	0	1	2	3
Table Operation	1	2	0	3
Hint	0	4	0	4

TABLE VII
TIME CONSUMPTION FOR EACH PART OF OFFLINE RULE LEARNING PROCESS

Part	PG-Strom	HeavyDB	Dask-sql
Generator	0.27h	0.69h	0.64h
Labeler (Query Plan)	234s	0.40h	-
Labeler (Program Trace)	-	-	0.73h
Parser	21s	4s	3s
Learner	48s	29s	22s
Sum	0.35h	1.10h	1.38h

Answer to RQ1: SQLaw discovers a total of 51 previously unknown GPU-related bugs in *PG-Strom*, *HeavyDB*, and *Dask-sql*, of which 37 have been confirmed by developers.

Time Consumption of Offline Rule Learning. SQLaw includes an additional offline rule learning process to learn differential execution rules compared with the existing approaches. To better understand the cost of offline rule learning, we measure its time consumption.

Table VII shows the time consumption for each part of the offline rule learning process in three GPU DBMSs under test. Although *PG-Strom* requires generating and labeling more query statements with configurations (explained in §IV), it achieves more efficient processing of query statements with configurations (*Generator* needs to verify the correctness of generated query statements with configurations via databases), thereby consuming less time. Overall, the time cost of offline rule learning for all three databases is under 1.4 hours. Notably, offline rule learning process is a one-time effort, unless there are major updates to GPU DBMSs under test. Despite its time cost, the generated differential execution rules can be reused during testing. It is marginally consumptive compared to 24-hour continuous testing as long as it improves bug-finding effectiveness and efficiency.

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

TABLE VIII
UNIQUE BUGS DETECTED IN THREE GPU DBMSS UNDER TEST WITHIN 24 HOURS

DBMS	SQLaw	SQLsmith*	TLP*	NoREC*
PG-Strom	22	10	7	5
HeavyDB	15	6	6	4
Dask-sql	14	7	4	6
Total	51	23	17	15
Improvement	-	122% ↑	200% ↑	240% ↑

TABLE IX
POWER AND MEMORY CONSUMPTION, INCLUDINGS AVERAGE CPU POWER, AVERAGE CPU MEMORY USAGE, AVERAGE GPU POWER AND AVERAGE GPU MEMORY USAGE

DBMS	PG-Strom	HeavyDB	Dask-sql
SQLaw	40W 457MB 58W 1222MB	32W 1130MB 79W 2551MB	62W 1407MB 81W 1980MB
SQLsmith*	19W 439MB 44W 1222MB	29W 685MB 77W 1986MB	62W 1324MB 80W 1980MB
TLP*	39W 442MB 53W 1222MB	30W 1060MB 74W2091MB	61W 1224MB 79W 1957MB
NoREC*	36W 434MB 55W 1222MB	31W 1012MB 72W 2412MB	61W 1306MB 79W 1965MB

C. Comparison With Baselines

GPU-related Bug Detection. For a more comprehensive evaluation of SQLaw, we compared it with the three state-of-the-art approaches with GPU DBMSs adaptation, including *SQLsmith**, which is good at detecting GPU crash bugs and GPU error bugs, *NoREC** and *TLP**, which are proficient at finding GPU logic bugs. We ran all of the approaches with a time budget of 24 hours. As shown in Table VIII, for *PG-Strom*, SQLaw detected 22 GPU-related bugs, while *SQLsmith**, *TLP** and *NoREC** only found 10, 7, and 5 bugs, respectively. For *HeavyDB*, SQLaw detected 15 GPU-related bugs, while *SQLsmith**, *TLP** and *NoREC** only discovered 6, 6, 4 bugs, respectively. For *Dask-sql*, SQLaw detected 14 GPU-related bugs, while *SQLsmith**, *TLP** and *NoREC** only detected 7, 4, 6 bugs, respectively. In total, the number of GPU-related bugs detected by SQLaw is 122% to 240% more than the one by the baseline approaches.

In terms of bug types, SQLaw found 11, 6 and 7 GPU crash and error bugs in *PG-Strom*, *HeavyDB* and *Dask-sql*, respectively. It covered 10, 6 and 7 of GPU crash and error bugs found by *SQLsmith**. SQLaw detected 11, 9 and 7 GPU logic bugs in *PG-Strom*, *HeavyDB* and *Dask-sql*, respectively. It is considerably more than 2 and 1 GPU logic bugs detected by *TLP** in *PG-Strom* and *HeavyDB*, and only 1 GPU logic bug detected by *NoREC** in *PG-Strom*. Furthermore, *TLP** cannot find any GPU logic bugs in *Dask-sql*. Neither can *NoREC** find any GPU logic bugs in either *HeavyDB* or *Dask-sql*. Overall, SQLaw outperforms corresponding state-of-the-art approaches for detecting crash and GPU error bugs as well as GPU logic bugs. These results demonstrate the superiority of SQLaw in terms of GPU-related bug detection.

Answer to RQ2: Compared with the existing state-of-the-art approaches with GPU DBMSs adaptation, SQLaw can detect 122% to 240% more GPU-related bugs in three GPU DBMSs under test.

Testing Time Budget for SQLaw. In most cases, GPU DBMS features remain stable, allowing rules learned during the offline process to be reused effectively. However, when new features are introduced, SQLaw must perform offline rule learning to adapt to these changes, which inevitably incurs additional time overhead. To ensure fair evaluation under such circumstances, we compare SQLaw with the baseline approaches by taking the time cost of offline rule learning into account. It should be noted that SQLaw can only begin bug detection after completing the offline process, causing it to initially lag behind the baselines. Nevertheless, it requires only 1 hour, 2 hours, and 2 hours to outperform the best baseline in *PG-strom*, *HeavyDB*, and *Dask-sql*, respectively. The corresponding offline rule learning durations are 0.35 hours, 1.1 hours, and 1.38 hours for these three systems.

Power and Memory Consumption. To quantify energy and resource consumption of SQLaw and the baselines during 24-hour testing, we used the *Intel RAPL* [54] interface to measure average CPU power, while utilizing the library *psutil* [55] to collect average CPU memory usage. Additionally, average GPU power and memory usage were monitored through the tool *py3nvml* [56].

As shown in Table IX, SQLaw demonstrates higher CPU and memory usage than baselines. Moreover, since SQLaw generates query statements with configurations that execute more effectively on expected devices than baselines, particularly GPU devices, SQLaw exhibits higher GPU power and memory consumption in the GPU DBMSs under test. While this results in slightly elevated resource demands across CPU, memory, and GPU, the trade-off is justified by SQLaw's superior bug detection capability, making the additional overhead worthwhile.

D. Ablation Study

Contribution of Offline Rule Learning. To evaluate the contribution of offline rule learning, which has mined differential execution rules, we compared the percentage of query statements executed on the expected devices under the configurations generated with and without differential execution rules. The former uses an *Interpreter* in §III-C to generate pairs of CPU-only and GPU-used query statements with configurations according to differential execution rules mined from offline rule learning. Meanwhile, the latter employs a *Generator* in §III-B1 to randomly generate query statements with pivotal configurations.

Table X shows that for *PG-Strom*, only 7.7% of query statement pairs with configurations without differential execution

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

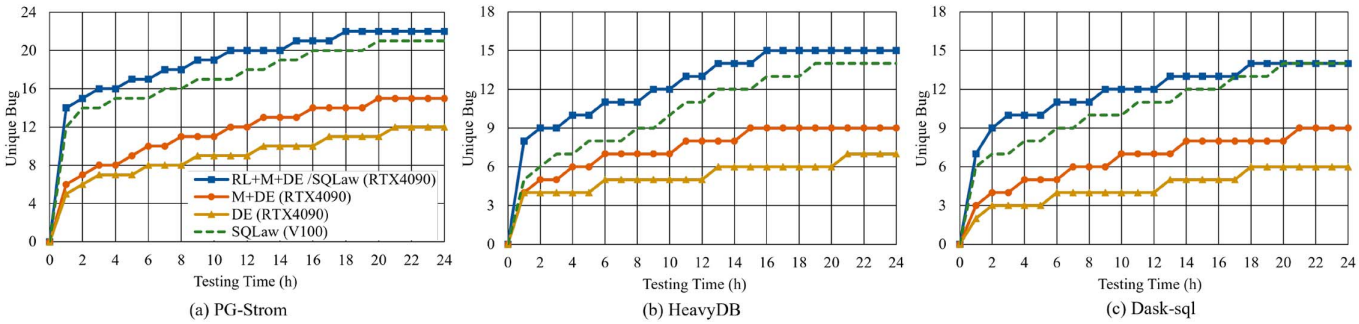


Fig. 14. 24-hour bug detection in three GPU DBMSs under test. *RL*: Rule Learning (differential execution rules), *M*: Mutation, *DE*: Differential Execution (differential testing on GPU DBMSs). These components are combined for the ablation study. *M+DE* and *DE* adopt the pivotal configuration approach detailed in § II-C to control CPU-only and GPU-used execution.

TABLE X
THE PERCENTAGE OF QUERY STATEMENTS WITH CONFIGURATIONS CAPABLE OF EXPECTED EXECUTION

DBMS	With Differential Execution Rule	Without Differential Execution Rule	Improvement
PG-Strom	40.1%	7.7%	420.8% ↑
HeavyDB	98.2%	79.8%	23.1% ↑
Dask-sql	85.9%	56.4%	52.3% ↑

rules are executed on the expected devices. However, this percentage increases to 40.1% when using differential execution rules, with the improvement of 420.8% over the one without differential execution rules. For *HeavyDB* and *Dask-sql*, the percentage of query statement pairs using differential execution rules increases by 23.1% and 52.3% respectively, compared to the one without differential execution rules.

The contribution of offline rule learning was also evaluated by comparing GPU-related bug detection performance over 24 hours, with and without differential execution rules, as represented by *RL+M+DE* and *M+DE* respectively in Fig. 14. The differential execution rules derived from offline rule learning enhance bug detection efficiency, increasing the number of detected GPU-related bugs by 1.47× to 1.67×.

Contribution of Mutation and Differential Execution Relation. To evaluate the contribution of the mutation in *Interpreter* and the differential execution relation in *Bug Detector*, we compared the performance of GPU-related bug detection over 24 hours with and without mutations, as represented by *M+DE* and *DE* in Fig. 14.

Fig. 14 indicates that the mutation enhances GPU-related bug detection, particularly in the case of *Dask-sql*. The mutation increases the detection number of GPU-related bugs by 1.25× to 1.50×. Additionally, the differential execution relation serves as an effective test oracle for GPU-related bug detection. It contributes from 42.9% to 54.5% of the total detected GPU-related bugs.

Comparison with Differential Testing. Over a 24-hour period, the differential testing approach, represented by *DE* in Fig. 14, detected 12, 7 and 6 GPU-related bugs in *PG-Strom*, *HeavyDB* and *Dask-sql*, respectively. In comparison, SQLaw detected 83%, 114%, and 133% more GPU-related bugs than this differential testing approach across three GPU DBMSs under test, respectively.

Answer to RQ3: The differential execution rules mined through offline rule learning can improve the effectiveness of generated query statements that are executed on the expected devices, with the improvement of 23.1% to 420.8% over the approach without the differential execution rules. It aids in the implementation of the test oracle for GPU-related bug detection.

E. Discussion

Threshold Selection. *Learner* in §III-B4 relies on three threshold values t_l, t_{fc} and t_{fg} to regulate the extraction of clause rules, CPU-only configuration rules and GPU-used configuration rules. These thresholds critically influence the quality of differential execution rules, which in turn determines the effectiveness of bug detection in later stages. Consequently, the optimal selection of these thresholds aims to maximize bug detection performance.

Following common practices in *association rule learning*, the threshold values for *support* and *confidence* are conventionally set within the range of 0.1 to 0.9 [20], [42]. Accordingly, we initialize the search space for all three threshold parameters within this range, using a step size of 0.2, which yields 125 possible combinations. We used each combination of the thresholds to derive differential execution rules, which in turn generate query statements for statistical evaluation of expected execution rates. For each combination, we select the top twenty rules (ranked by expected execution rates) for 3-hour GPU-related bug detection. The optimal threshold combination (t_l: 0.1, t_{fc}: 0.9, t_{fg}: 0.5) is ultimately chosen, as it demonstrates the best performance in practical bug detection scenarios.

Impact of GPU Hardware Devices. To evaluate the impact of GPU hardware configuration on SQLaw's performance, we selected a NVIDIA Tesla V100 (16GB) GPU [50] for experiments, holding all other hardware configurations constant except for the GPU, and compared it with a NVIDIA GeForce

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

TABLE XI
TECHNICAL SPECIFICATIONS OF NVIDIA GEFORCE RTX 4090 AND TESLA V100 GPU

GPU	Architecture	Memory Size & Bandwidth	Single-Precision Performance
RTX 4090	Ada Lovelace	24GB 1.01TB/s	83TFLOPS
Tesla V100	Volta	16GB 900GB/s	14TFLOPS

```
-- GPU Crash Bug
SET pg_strom.enabled = ON;
SELECT MIN(t2.c0) FROM t2 LEFT OUTER JOIN
        (SELECT c0 FROM t1) AS sub0
       ON (t2.c0>0) GROUP BY (t2.c0) HAVING BOOL_AND(TRUE);
-- GPU-used Result
> server closed the connection unexpectedly.
```

```
-- GPU Error Bug
SET pg_strom.enabled = ON;
SELECT t2.c0 FROM t1 RIGHT OUTER JOIN t2
                    ON (CAST(0.1 AS MONEY) IS NULL);
-- GPU-used Result
> ERROR: gpu_service.c:1600 failed on
            cuEventSynchronize: CUDA_ERROR_ILLEGAL_ADDRESS.
```

```
-- GPU Logic Bug
SET pg_strom.enabled = OFF | ON;
SELECT * FROM t1, t0 WHERE((t1.c1)
                    IS NOT DISTINCT FROM (t0.c1)) IS TRUE;
-- CPU-only Result          | -- GPU-used Result
>   c0 | c1 | c0 | c1        >   c0 | c1 | c0 | c1
   ----+----+----+----          ----+----+----+----
     1 | 0  | 1  | 0             1 | 0  | 1  | 0
     1 | 0  | 1  | 0             (1 row)
   (2 rows)                                          Different
```

Fig. 15. Examples of GPU-related bugs in *PG-Strom*.

RTX 4090 (24GB) GPU [49]. Technical specifications of these GPUs are presented in Table XI.

GPU hardware configurations primarily affect the execution speed of SQL statements in GPU DBMSs under test, thereby influencing SQLaw's efficiency in GPU-related bug detection. As shown in Fig. 14, because the overall performance of the RTX 4090 GPU surpasses that of the Tesla V100, SQLaw demonstrates higher GPU-related bug detection efficiency on the RTX 4090 than on the Tesla V100.

Case Study. To further understand the usefulness of SQLaw, we discuss five representative GPU-related bugs found by SQLaw in three GPU DBMSs under test.

Fig. 15 presents three types of GPU-related bugs in *PG-Strom*. To analyze the root cause of these bugs, we review bug-fixing commits made by the developers. The GPU crash bug is caused by a wrong status check around the *HAVING* clause in the *gpu_preagg.c* file, and the *JOIN* clause error in the *gpu_join.c* file leads to this GPU error bug. Due to effective mining of correlations between configurations, clauses and expected execution devices, SQLaw can generate test cases to trigger GPU-related bugs. Additionally, the GPU logic bug caused by a boolean expression *NOT* referencing to an uninitialized variable in the *xpu_common.cu* file. It suggests that the expression mutation in SQLaw can help detect more hidden GPU-related bugs.

The hint `/*+ keep_result */` is used to add result set of the input query to the result set cache and should not influence the correctness of query results [37]. However, Fig. 16

```
-- GPU Logic Bug
ALTER SESSION SET EXECUTOR_DEVICE = 'CPU'| 'GPU';
SELECT /*+ keep_result */ * FROM t0 LIMIT 1 OFFSET 1;
 -- CPU-only Result       | -- GPU-used Result
> No rows returned.        > c0
                             0.10000000149011612      Different
```

Fig. 16. An example of GPU-related bugs in *HeavyDB*.

```
# GPU_Error_Bug.py
import dask.dataframe as dd
from dask_sql import Context
c = Context()
c.create_table(…)… # create GPU table(t0_gpu,t1_gpu,t3_gpu)
 …
print('GPU Result:')
result= c.sql("SELECT (('A')!=(t1_gpu.c0))
                    FROM t0_gpu, t1_gpu, t3_gpu").compute()
print(result)

# python GPU_Error_Bug.py
> GPU Result:
 Traceback (most recent call last): …
  … ValueError: Metadata inference failed in `ne`. …
 File "…/cudf/core/mixins/mixin_factory.py", line 11, …
 File "…/nvtx/nvtx.py", line 101, …
 File "…/cudf/core/frame.py", line 1750, …
 …
```

Fig. 17. An example of GPU-related bugs in *Dask-sql*.

presents a query statement with this hint that leads to different results when executed on CPU and GPU devices in *HeavyDB*. Thanks to the ability to extract diverse differential execution rules from configurations, hints and clauses in randomly generated datasets, SQLaw can find such GPU logic bug. Meanwhile, the existing baseline approaches which leverage the pivotal configurations cannot find such bug, because the hint `/*+ keep_result */` is not a pivotal configuration and will not be used for the generated query statements.

Fig. 17 presents a GPU error bug in *Dask-sql* with some paths related to *cudf* [39] and *nvtx* [57] in the traceback messages. This indicates that the error occurred during the invocation of GPU-related libraries.

In summary, thanks to the effective mining of correlations between configurations, hints, clauses, and expected execution devices via offline differential execution rules, and online expression mutation, SQLaw can efficiently generate test cases to trigger hidden GPU-related bugs.

Future Work. Although the mutation strategy in SQLaw improves GPU-related bug detection, as verified in the ablation study, it is currently implemented at the syntax level. In future work, we plan to study semantic-level mutation strategies. Furthermore, since SQLaw primarily focuses on correctness testing for GPU DBMSs in serial execution, we will expand our work to detect concurrent execution bugs. We also plan to deploy SQLaw on additional GPU DBMSs to further enhance the generalizability of our approach.

VI. RELATED WORK

Although SQLaw is a unique GPU DBMSs bug detection system, it is also inspired by several areas in the literature,

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

including GPU DBMSs, DBMSs testing, fuzzing, metamorphic testing, differential testing, GPU-related testing and rule learning.

GPU DBMSs. Existing efforts related to GPU DBMSs fall into three main categories, including the extension of traditional DBMSs, in-memory GPU DBMSs, and non-memory GPU DBMSs. *PG-Strom* [29] and *DB2 BLU* [58] are the representatives of the extensions of traditional DBMSs, which are based on *PostgreSQL* [59] and *DB2* [60], respectively. These extensions work as a plugin of the existing DBMSs and leverage GPU to accelerate SQL workloads for data analytics. The in-memory GPU DBMSs, including *HeavyDB* [32], *OmniSci* [30], *MapD* [31], *Brytlyt* [22], and *Kinetica* [23], etc., is designed to utilize the full potential performance of the GPU and increases the speed of data processing by having all the data reside in memory. The non-memory GPU DBMSs, including *Dask-sql* [26], *BlazingSQL* [25], *SQreamDB* [24], etc., leverages the GPU to perform all or most of the database relational operations, allowing for the analysis of larger data volumes. In a word, these GPU DBMSs under test are well represented. With the emergence of more and more GPU DBMSs, we believe our technique will provide important values in the reliability of these DBMSs.

DBMS Testing. Recent research studies on DBMS testing focus on various aspects of DBMSs. In terms of DBMS types, significant research efforts have been devoted to relational DBMSs [14], [52], [53], [61], [62], [63], [64], graph DBMSs [13], [65], [66], and RDF stores [67]. Different from the prior studies, SQLaw focuses on detecting GPU-related bugs in GPU DBMSs, a brandly new type of DBMSs in terms of the execution processor unit.

With respect to methodologies for DBMS testing, existing research can be primarily categorized into three groups: DBMS fuzzing [68], metamorphic testing [69] and differential testing [19].

DBMS Fuzzing. DBMS fuzzing approaches generate test cases and verify whether the DBMS operates normally during executing them to identify bugs [68]. *SQLsmith* [70] is a leading grammar-based DBMS fuzzer that randomly generates SQL queries to test various DBMSs. *SQUIRREL* [71], *Griffin* [72], *LEGO* [63], *DynSQL* [73], *SQLRight* [74], and *WINGFUZZ* [75] are mutation-based fuzzers that utilize coverage information to guide mutation from original queries to new ones, thereby generating diverse queries to test DBMSs.

In contrast to these approaches, we employ an AST-based fuzzer to randomly generate SQL datasets for mining differential execution rules. This helps SQLaw to generate query statements with configurations capable of expected execution, thereby detect GPU-related bugs more effectively.

Metamorphic Testing. Many studies [52], [53], [61], [62], [64], [65], [66] employ metamorphic testing to evaluate DBMSs. Metamorphic testing verifies SQL statement results against predefined rules to detect bugs [69]. *TLP* (Ternary Logic Partitioning) is an effective metamorphic relation for detecting logic bugs in both relational DBMSs [53] and graph DBMSs [65]. To uncover logic bugs in relational DBMSs, *NoREC* [52] verifies the equivalence between original queries and their non-optimized translations; *PINOLO* [62] employs the approximation relation; *TQS* [61] utilizes join optimization relations; and *EET* [64] constructs equivalent transformations through expression-level manipulation. Additionally, *GAMERA* [66] leverages graph-aware metamorphic relations to detect logic bugs in graph DBMSs. These approaches rely on customized relations or rules that are less tailored to the characteristics of GPU DBMSs. This limits their direct applicability in effectively detecting GPU-related bugs.

Differential Testing. Differential testing compares the results of identical test cases executed on different software systems or versions to identify inconsistencies, correctness bugs and performance issues [19]. This technique is widely used in DBMS testing. For example, *DT2* [15] identifies transaction discrepancies across multiple DBMSs by comparing the results of identical transactions. To detect correctness bugs, *Grand* [13] and *RD2* [67] employ randomized differential testing for multiple graph DBMSs and RDF stores, respectively; *THANOS* [14] compares test cases on differential storage engines; and *DQP* [16] leverages differential query plans for simple yet effective bug detection. *APOLLO* [76] detects performance issues by comparing SQL execution speeds across different DBMS versions. Moreover, *Mozi* [77] uncovers both performance and correctness bugs by altering DBMS configurations to create varied system states for comparison.

Beyond DBMS testing, differential testing is widely applied to test various software applications, such as ORM frameworks [78], compilers [79], JVMs [80], JavaScript engines [81] and search engines [82].

Unlike previous studies, SQLaw can be considered a novel extension of differential testing, since it combines rule learning, mutation, and differential execution relation for more efficient testing. Different from conventional differential testing that compares across multiple systems or versions, SQLaw compares differential execution between CPU and GPU devices within the same GPU DBMS for correctness bugs, thereby circumventing the limitations imposed by differences in dialects or version-specific features.

GPU-related Testing. Several studies focus on GPU kernel verification, such as *GKLEE* [83], *cuCatch* [10] and *CLTestCheck* [84]. These tools detect data races, identify memory safety errors and verify the correctness of GPU source programs. Additionally, some research addresses GPU-related defects in software systems. For instance, *DarthShader* [85] employs fuzzing to detect security bugs in WebGPU shader translators and compilers, while *TensorScope* [18] identifies GPU-related vulnerabilities through differential testing of cross deep learning framework APIs. *D3* [17] trains the same model under different distributed settings and crosschecks the model outputs to detect bugs in distributed deep learning systems. The prior studies do not involve GPU DBMS testing and inspire our exploration of this new domain.

Rule Learning. Rule learning is the process of discovering and extracting patterns, relationships, or rules from data [86]. Rule learning is good at describing significant patterns in the given dataset in terms of rules [41]. Prior studies have shown its effectiveness in the domain and application of knowledge-based

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

reasoning [87], [88], web threat analysis [89], and synthesizing configuration file specifications [42]. In this work, we aim at discovering underlying correlations between configurations, clauses and execution devices, which is similar to the prior successful application of rule learning. Thus, we propose to apply rule learning to SQLaw.

VII. Conclusion

We have presented a novel GPU DBMS testing framework termed SQLaw. The basic idea of the SQLaw is to combine offline rule learning and an online interpreter with mutation for efficient detection of GPU-related bugs. The evaluation results show that it has discovered 51 previously unknown GPU-related bugs, and 37 have already been confirmed or fixed by developers. We expect that SQLaw will enhance the robustness and prosperity of GPU DBMSs.

Acknowledgment

We thank anonymous reviewers for their insightful comments.

References

[1] A. Shanbhag, S. Madden, and X. Yu, "A study of the fundamental performance characteristics of GPUs and CPUs for database analytics," in *Proc. SIGMOD'20*, 2020, pp. 1617–1632.
[2] V. Rosenfeld, S. Breß, and V. Markl, "Query processing on heterogeneous CPU/GPU systems," *CSUR'22*, vol. 55, no. 1, pp. 1–38, 2022.
[3] C. Lutz, S. Breß, S. Zeuch, T. Rabl, and V. Markl, "Triton join: efficiently scaling to a large join state on GPUs with fast interconnects," in *Proc. SIGMOD'22*, 2022, pp. 1017–1032.
[4] J. Cao, R. Sen, M. Interlandi, J. Arulraj, and H. Kim, "GPU database systems characterization and optimization," in *Proc. VLDB'23*, vol. 17, no. 3, 2023, pp. 441–454.
[5] J. Krueger, M. Grund, I. Jaeckel, A. Zeier, and H. Plattner, "Applicability of GPU computing for efficient merge in in-memory databases," in *Proc. VLDB'11*, 2011, pp. 19–26.
[6] HEAVY.AI, "HEAVY.AI For Your Industry: Learn how accelerated analytics can transform your enterprise." Accessed: Dec. 20, 2024. [Online]. Available: https://www.heavy.ai/industries
[7] SQreamDB, "SQreamDB: Case Studies." Accessed: Dec. 20, 2024. [Online]. Available: https://sqream.com/resources/?category=case-studies
[8] K. D. Inc., "Kinetica for Automotive: Real-time analytics from vehicle sensor data." Accessed: Dec. 20, 2024. [Online]. Available: https://www.kinetica.com/solutions/automotive/
[9] Brytlyt, "Brytlyt for Energy, Oil and Gas: Extract the value from data to boost operation efficiency." 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://brytlyt.io/oil-and-gas/
[10] M. Tarek Ibn Ziad, S. Damani, A. Jaleel, S. W. Keckler, and M. Stephenson, "CuCatch: A debugging tool for efficiently catching memory safety violations in CUDA applications," in *Proc. PLDI*, vol. 7, 2023, pp. 124–147.
[11] M. Wu, Y. Ouyang, H. Zhou, L. Zhang, C. Liu, and Y. Zhang, "Simulee: Detecting CUDA synchronization bugs via memory-access modeling," in *Proc. ICSE*, 2020, pp. 937–948.
[12] F. R. Monteiro, E. H. d S. Alves, I. S. Silva, H. I. Ismail, L. C. Cordeiro, and E. B. de Lima Filho, "ESBMC-GPU a context-bounded model checking tool to verify CUDA programs," *Sci. Comput. Program.*, vol. 152, pp. 63–69, Jan. 2018.
[13] Y. Zheng et al., "Finding bugs in gremlin-based graph database systems via randomized differential testing," in *Proc. ISSTA*, 2022, pp. 302–313.
[14] Y. Fu et al., "THANOs: DBMs bug detection via storage engine rotation based differential testing," in *Proc. ICSE*, 2025, pp. 1–12.
[15] Z. Cui et al., "Differentially testing database transactions for fun and profit," in *Proc. ASE*, 2022, pp. 1–12.
[16] J. Ba and M. Rigger, "Keep it simple: Testing databases via differential query plans," in *Proc. SIGMOD*, 2024, pp. 38–52.
[17] J. Wang et al., "D3: Differential testing of distributed deep learning with model generation," *IEEE Trans. Softw. Eng.*, vol. 51, no. 1, pp. 38–52, Jan. 2025.
[18] Z. Deng, G. Meng, K. Chen, T. Liu, L. Xiang, and C. Chen, "Differential testing of cross deep learning framework {APIs}: Revealing inconsistencies and vulnerabilities," in *Proc. USENIX Secur.*, 2023, pp. 7393–7410.
[19] W. M. McKeeman, "Differential testing for software," *Digit. Tech. J.*, vol. 10, no. 1, pp. 100–107, 1998.
[20] R. Agrawal, T. Imieliński, and A. Swami, "Mining association rules between sets of items in large databases," in *Proc. SIGMOD*, 1993, pp. 207–216.
[21] R. Agrawal and R. Srikant et al., "Fast algorithms for mining association rules," in *Proc. VLDB'94*, vol. 1215, 1994, pp. 487–499.
[22] "Brytlyt white paper: Speed of thought analytics at scale," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://www.brytlyt.com/
[23] "Kinetica on-premises documentation," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://docs.kinetica.com/
[24] "Sqream DB architecture whitepaper: A next generation GPU powered analytics SQL database," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://sqream.com/
[25] BlazingSQL, "BlazingSQL," 2021. Accessed: Dec. 20, 2024. [Online]. Available: https://github.com/BlazingDB/blazingsql
[26] Dask, "Dask-sql," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://github.com/dask-contrib/dask-sql
[27] K. Zhang et al., "Hetero-DB: next generation high-performance database systems by best utilizing heterogeneous computing and storage resources," *J. Comput. Sci. Technol.*, vol. 30, no. 4, pp. 657–678, 2015.
[28] R. Lee et al., "The art of balance: A rateupdb™ experience of building a CPU/GPU hybrid database product," in *Proc. VLDB*, vol. 14, 2021, pp. 2999–3013.
[29] K. Kohei, "PG-Strom," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://github.com/heterodb/pg-strom
[30] OmniSci, "OmniSci technical white paper: GPU-accelerated analytics-big data analytics at speed and SCAL," 2020. Accessed: Dec. 20, 2024. [Online]. Available: https://www.omnisci.com/platform
[31] C. Root and T. Mostak, "MAPD: A GPU-powered big data analytics and visualization platform," in *Proc. SIGGRAPH*, 2016, pp. 1–2. Accessed: Dec. 20, 2024.
[32] HEAVY.AI, "HeavyDB," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://github.com/heavyai/heavydb
[33] "DB-engines ranking," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://db-engines.com/en/ranking
[34] I. HeteroDB, "PG-Strom manual," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://heterodb.github.io/pg-strom/
[35] C. Lattner and V. Adve, "Llvm: A compilation framework for lifelong program analysis & transformation," *CGO'04*, 2004, pp. 75–86.
[36] NVIDIA, "Parallel thread execution ISA version 8.3," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://docs.nvidia.com/cuda/parallel-thread-execution/index.html
[37] HEAVY.AI, "HEAVY.AI documentation," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://docs.heavy.ai/
[38] RAPIDS Development Team, "RAPIDS: Collection of libraries for end to end GPU data science," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://rapids.ai
[39] RAPIDS Development Team, "cuDF - GPU DataFrames," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://github.com/rapidsai/cudf
[40] N. Braun, "Dask-sql Documentation," 2024. Accessed: Dec. 20, 2024. [Online]. Available: https://dask-sql.readthedocs.io/en/latest/index.html
[41] J. Fürnkranz and T. Kliegr, "A brief overview of rule learning," in *Proc. Int. Symp. Rules Rule Markup Lang. Semantic Web*, 2015, pp. 54–69.
[42] M. Santolucito, E. Zhai, R. Dhodapkar, A. Shim, and R. Piskac, "Synthesizing configuration file specifications with association rule learning," in *Proc. OOPSLA*, vol. 1, 2017, pp. 1–20.
[43] V. Leis, A. Gubichev, A. Mirchev, P. Boncz, A. Kemper, and T. Neumann, "How good are query optimizers, really?" in *Proc. VLDB*, vol. 9, no. 3, 2015, pp. 204–215.
[44] P. S. Foundation, "Trace — Trace or track Python statement execution," 2024. [Online]. Available: https://docs.python.org/zh-cn/3/library/trace.html
[45] T. Parr, "ANTLR v4," 2023. [Online]. Available: https://github.com/antlr/antlr
[46] T. Parr and K. Fisher, "Ll (*) the foundation of the ANTLR parser generator," *ACM Sigplan Notices*, vol. 46, no. 6, pp. 425–436, 2011.

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.

[47] M. Rigger, "SQLancer," 2024. [Online]. Available: https://github.com/sqlancer/sqlancer
[48] A. Zeller, *Why Programs Fail: A Guide to Systematic Debugging*, San Francisco, CA, USA: Morgan Kaufmann Publishers Inc. Oct 2005.
[49] NVIDIA, "NVIDIA GeForce RTX 4090 datasheet," 2024. [Online]. Available: https://www.nvidia.com/en-us/geforce/graphics-cards/40-series/rtx-4090/
[50] NVIDIA, "NVIDIA Tesla V100 datasheet," 2018. [Online]. Available: https://images.nvidia.com/content/technologies/volta/pdf/tesla-volta-v100-datasheet-letter-fnl-web.pdf
[51] A. Seltenreich, "SQLsmith: A random SQL query generator," Mar. 2023. Accessed Dec-2022. [Online]. Available: https://github.com/anse1/sqlsmith
[52] M. Rigger and Z. Su, "Detecting optimization bugs in database engines via non-optimizing reference engine construction," in *Proc. ESEC/FSE*, 2020, pp. 1140–1152.
[53] M. Rigger and Z. Su, "Finding bugs in database systems via query partitioning," *Oopsla'20,* 2020.
[54] K. N. Khan, M. Hirki, T. Niemi, J. K. Nurminen, and Z. Ou, "RAPL in action: Experiences in using RAPL for power measurements," in *Proc. TOMPECS*, vol. 3, no. 2, 2018, pp. 1–26.
[55] G. Rodola, "psutil library," 2025. [Online]. Available: https://pypi.org/project/psutil/
[56] F. Cotter, "py3nvml library," 2025. [Online]. Available: https://pypi.org/project/py3nvml/
[57] NVIDIA, "NVTX (NVIDIA Tools Extension Library)," 2024. [Online]. Available: https://github.com/NVIDIA/NVTX
[58] S. Meraji et al., "Towards a hybrid design for fast query processing in DB2 with Blu acceleration using graphical processing units: A technology demonstration," in *Proc. SIGMOD*, 2016, pp. 1951–1960.
[59] PostgreSQL, "PostgreSQL: The world's most advanced open source relational database," 2023. [Online]. Available: https://www.postgresql.org/
[60] IBM, "IBM Db2 database," 2023. [Online]. Available: https://www.ibm.com/products/db2/database
[61] X. Tang, S. Wu, D. Zhang, F. Li, and G. Chen, "Detecting logic bugs of join optimizations in DBMs," in *Proc. SIGMOD*, vol. 1, no. 1, 2023, pp. 1–26.
[62] Z. Hao et al., "PINOLO: Detecting logical bugs in database management systems with approximate query synthesis," in *Proc. USENIX ATC,* 2023, pp. 345–358.
[63] J. Liang et al., "Sequence-oriented DBMs fuzzing," in *Proc. ICDE*, 2023, pp. 668–681.
[64] Z.-M. Jiang and Z. Su, "Detecting logic bugs in database engines via equivalent expression transformation," in *Proc. USENIX OSDI*, 2024, pp. 821–835.
[65] M. Kamm, M. Rigger, C. Zhang, and Z. Su, "Testing graph database engines via query partitioning," in *Proc. ISSTA*, 2023, pp. 140–149.
[66] Z. Zhuang, P. Li, P. Ma, W. Meng, and S. Wang, "Testing graph database systems via graph-aware metamorphic relations," in *Proc. VLDB*, vol. 17, no. 4, 2023, pp. 836–848.
[67] R. Yang, Y. Zheng, L. Tang, W. Dou, W. Wang, and J. Wei, "Randomized differential testing of RDF stores," in *Proc. ICSE-Companion*, 2023, pp. 136–140.
[68] X. Zhu, S. Wen, S. Camtepe, and Y. Xiang, "Fuzzing: A survey for roadmap," *ACM Comput. Surv.*, vol. 54, no. 11s, pp. 1–36, 2022.
[69] T. Y. Chen et al., "Metamorphic testing: A review of challenges and opportunities," *ACM Comput. Surv.*, vol. 51, no. 1, pp. 1–27, 2018.
[70] M. Jibson, "SQLsmith: randomized SQL testing in CockroachDB," *Cockroach Labs*, 2019.
[71] R. Zhong, Y. Chen, H. Hu, H. Zhang, W. Lee, and D. Wu, "Squirrel: Testing database management systems with language validity and coverage feedback," in *Proc. CCS*, 2020, pp. 955–970.
[72] J. Fu, J. Liang, Z. Wu, M. Wang, and Y. Jiang, "Griffin: Grammar-free DBMs fuzzing," in *Proc. ASE*, 2022, pp. 1–12.
[73] Z.-M. Jiang, J.-J. Bai, and Z. Su, "DynSQL: Stateful fuzzing for database management systems with complex and valid SQL query generation," in *Proc. USENIX Secur.*, 2023, pp. 4949–4965.
[74] Y. Liang, S. Liu, and H. Hu, "Detecting logical bugs of DBMS with coverage-based guidance," in *Proc. USENIX Secur.*, 2022, pp. 4309–4326.
[75] J. Liang, Z. Wu, J. Fu, Y. Bai, Q. Zhang, and Y. Jiang, "WingFuzz: Implementing continuous fuzzing for DBMSs," in *Proc. USENIX ATC*, 2024, pp. 479–492.
[76] J. Jung, H. Hu, J. Arulraj, T. Kim, and W. Kang, "APOLLO: Automatic detection and diagnosis of performance regressions in database systems," in *Proc. VLDB*, vol. 13, no. 1, 2019, pp. 57–70.
[77] J. Liang, Z. Wu, J. Fu, M. Wang, C. Sun, and Y. Jiang, "MOZI: Discovering DBMs bugs via configuration-based equivalent transformation," in *Proc. ICSE*, 2024, pp. 1–12.
[78] T. Sotiropoulos, S. Chaliasos, V. Atlidakis, D. Mitropoulos, and D. Spinellis, "Data-oriented differential testing of object-relational mapping systems," in *Proc. ICSE*, 2021, pp. 1535–1547.
[79] M. A. Gulzar, Y. Zhu, and X. Han, "Perception and practices of differential testing," in *Proc. ICSE-SEIP*, 2019, pp. 71–80.
[80] Y. Chen, T. Su, and Z. Su, "Deep differential testing of JVM implementations," in *Proc. ICSE*, 2019, pp. 1257–1268.
[81] I. Lima, J. Silva, B. Miranda, G. Pinto, and M. d'Amorim, "Exposing bugs in JavaScript engines through test transplantation and differential testing," *Softw. Qual. J.*, vol. 29, pp. 129–158, 2021.
[82] E. T. Barr, M. Harman, P. McMinn, M. Shahbaz, and S. Yoo, "The oracle problem in software testing: A survey," *IEEE Trans. Softw. Eng.*, vol. 41, no. 5, pp. 507–525, May 2015.
[83] G. Li, P. Li, G. Sawaya, G. Gopalakrishnan, I. Ghosh, and S. P. Rajan, "GKLEE: Concolic verification and test generation for GPUs," in *Proc. PPoPP*, 2012, pp. 215–224.
[84] C. Peng, "On the correctness of GPU programs," in *Proc. ISSTA*, 2019, pp. 443–447.
[85] L. Bernhard, N. Schiller, M. Schloegel, N. Bars, and T. Holz, "DarthShader: Fuzzing WebGPU shader translators & compilers," in *Proc. CCS*, 2024, pp. 690–704.
[86] J. Fürnkranz, D. Gamberger, and N. Lavrač, *Foundations of Rule Learning*. New York, NY, USA: Springer Sci. & Bus. Media, 2012.
[87] L. Raedt, A. Dries, I. Thon, G. Broeck, and M. Verbeke, "Inducing probabilistic relational rules from probabilistic examples," in *Proc. IJCAI*, Jul. 2015, pp. 1835–1842.
[88] F. Yang, Z. Yang, and W. W. Cohen, "Differentiable learning of logical rules for knowledge base reasoning," in *Proc. NeurIPS*, vol. 30, 2017, pp. 2316–2325.
[89] R. Rawat, R. K. Chakrawarti, A. S. A. Raj, G. Mani, K. Chidambarathanu, and R. Bhardwaj, "Association rule learning for threat analysis using traffic analysis and packet filtering approach," *Int. J. Inf. Technol.*, Aug. 2023, pp. 3245–3255, doi: 10.1007/s41870-023-01353-0.



Jiaxin Hu is currently working toward the Ph.D. degree with the School of Informatics, Xiamen University, China. His research focuses on intelligent software security and testing, performance optimization and deployment of heterogeneous software systems. He has published several papers in top-tier journals, including IEEE TRANSACTIONS ON SOFTWARE ENGINEERING and iScience.



Rongxin Wu received the Ph.D. degree from HKUST, in 2017. Currently, he is an Associate Professor with the Department of Computer Science and Technology, School of Informatics, Xiamen University. His research interests include program analysis, software security, and mining software repository. His research work has been regularly published in top conferences and journals in the research communities of program languages and software engineering, including POPL, PLDI, ATC, ICSE, FSE, ISSTA, ASE, and IEEE TRANSACTIONS ON SOFTWARE ENGINEERING and so on. He has served as a Reviewer in reputable international journals and a Program Committee Member in several international conferences (FSE'25, ISSTA'25, SANER'25, FSE'24, ISSTA'24, ASE'23, SANER'23, and ASE 2021 and so on). He works as a member of Xiamen Key Laboratory of Intelligent Storage and Computing, Xiamen University. He is a two-time recipient of the ACM SIGSOFT Distinguished Paper Award. For more information, see https://wurongxin1987.github.io/ wurongxin.xmu.edu.cn/

Authorized licensed use limited to: Xiamen University. Downloaded on October 10,2026 at 18:00:30 UTC from IEEE Xplore. Restrictions apply.